Architecture 1001: x86-64 Assembly Boolean instructions

Xeno Kovah – 2021
xeno@darkmentor.com

All materials is licensed under a Creative Commons “Share Alike” license.

- http://creativecommons.org/licenses/by-sa/4.0/

You are free:

to Share — to copy, distribute and transmit the work

to Remix — to adapt the work

Under the following conditions:

Attribution — You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

Attribution condition: You must indicate that derivative work
"Is derived from Xeno Kovah's 'Architecture 1001: x86-64 Assembly’ class, available at https://ost.fyi"

BooleanBecause.c

```c
#include <stdio.h>
#define uint64 unsigned long long

unsigned long long main() {
    unsigned int i = 0x50da;
    unsigned int j = 0xc0ffee;
    uint64 k = 0x7ee707a11ed;
    k ^= ~(i & j) | 0x7ab00;
    return k;
}
```

```
main:
0000000140001000  sub  rsp,18h
0000000140001004  mov  dword ptr [rsp+4],50DAh
000000014000100C  mov  dword ptr [rsp],0C0FFEEh
0000000140001013  mov  rax,7EE707A11EDh
000000014000101D  mov  qword ptr [rsp+8],rax
0000000140001022  mov  eax,dword ptr [rsp]
0000000140001025  mov  ecx,dword ptr [rsp+4]
0000000140001029  and  ecx,eax
000000014000102B  mov  eax,ecx
000000014000102D  not  eax
000000014000102F  or   eax,7AB00h
0000000140001034  mov  eax,eax
0000000140001036  mov  rcx,qword ptr [rsp+8]
000000014000103B  xor  rcx,rax
000000014000103E  mov  rax,rcx
0000000140001041  mov  qword ptr [rsp+8],rax
0000000140001046  mov  rax,qword ptr [rsp+8]
000000014000104B  add  rsp,18h
000000014000104F  ret
```

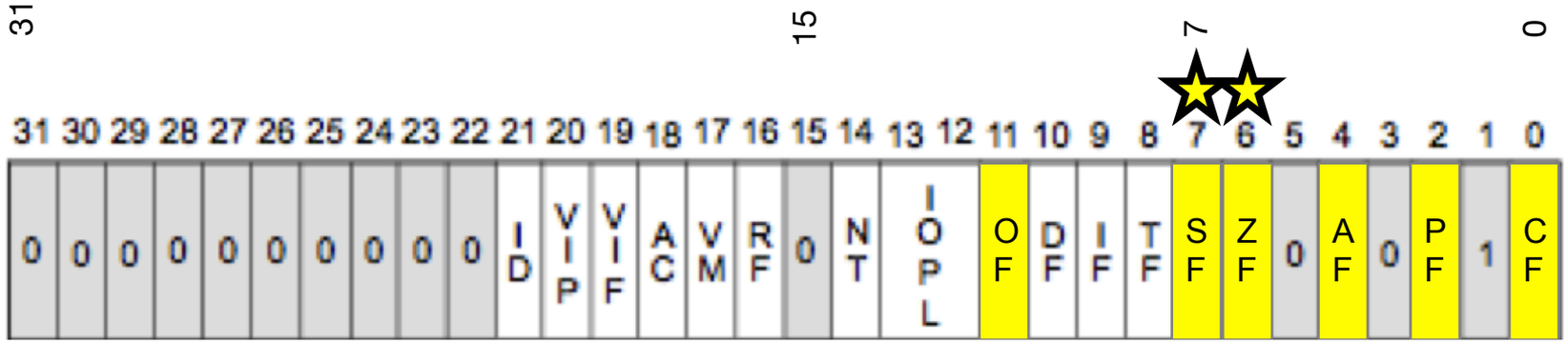
BooleanBecause.c

```c
#include <stdio.h>
#define uint64 unsigned long long

unsigned long long main() {
    unsigned int i = 0x50da;
    unsigned int j = 0xc0ffee;
    uint64 k = 0x7ee707a11ed;
    k ^= ~(i & j) | 0x7ab00;
    return k;
}
```

```
main:
0000000140001000   sub   rsp,18h
0000000140001004   mov   dword ptr [rsp+4],50DAh
000000014000100C   mov   dword ptr [rsp],0C0FFEEh
0000000140001013   mov   rax,7EE707A11EDh
000000014000101D   mov   qword ptr [rsp+8],rax
0000000140001022   mov   eax,dword ptr [rsp]
0000000140001025   mov   ecx,dword ptr [rsp+4]
0000000140001029   and   ecx,eax
000000014000102B   mov   eax,ecx
000000014000102D   not   eax
000000014000102F   or    eax,7AB00h
0000000140001034   mov   eax,eax
0000000140001036   mov   rcx,qword ptr [rsp+8]
000000014000103B   xor   rcx,rax
000000014000103E   mov   rax,rcx
0000000140001041   mov   qword ptr [rsp+8],rax
0000000140001046   mov   rax,qword ptr [rsp+8]
000000014000104B   add   rsp,18h
000000014000104F   ret
```

This Class



- Boolean operations modify most of the yellow status flags

X Alignment Check (AC)
X Virtual-8086 Mode (VM)
X Resume Flag (RF)
X Nested Task (NT)
X I/O Privilege Level (IOPL)
S Overflow Flag (OF)
C Direction Flag (DF)
X Interrupt Enable Flag (IF)
X Trap Flag (TF)
S Sign Flag (SF)
S Zero Flag (ZF)
S Auxiliary Carry Flag (AF)
S Parity Flag (PF)
S Carry Flag (CF)

S Indicates a Status Flag
C Indicates a Control Flag
X Indicates a System Flag

Reserved bit positions. DO NOT USE.
Always set to values previously read.

Figure 3-8. EFLAGS Register

AND - Bitwise AND

- C binary operator "&" (not &&, that's *logical* AND)
- Destination operand can be r/mX or register
- Source operand can be r/mX or register or immediate (No source *and* destination as r/mXs)

and al, bl

	00110011b (al - 0x33)
AND	01010101b (bl - 0x55)
result	00010001b (al - 0x11)

and al, 0x42

	00110011b (al - 0x33)
AND	01000010b (imm - 0x42)
result	00000010b (al - 0x02)

OR - Bitwise OR

- C binary operator "|" (not ||, that's *logical* OR)
- Destination operand can be r/mX or register
- Source operand can be r/mX or register or immediate (No source *and* destination as r/mXs)

or al, bl

	00110011b (al - 0x33)
OR	01010101b (bl - 0x55)
result	01110111b (al - 0x77)

or al, 0x42

	00110011b (al - 0x33)
OR	01000010b (imm - 0x42)
result	01110011b (al - 0x73)

XOR - Bitwise Exclusive OR

- C binary operator "^"
- Destination operand can be r/mX or register
- Source operand can be r/mX or register or immediate (No source *and* destination as r/mXs)
- FYI XOR is commonly used to zero a register, by XORing it with itself, because it's faster than a MOV

xor al, al

	00110011b (al - 0x33)
XOR	00110011b (al - 0x33)
result	00000000b (al - 0x00)

xor al, 0x42

	00110011b (al - 0x33)
OR	01000010b (imm - 0x42)
result	01110001b (al - 0x71)

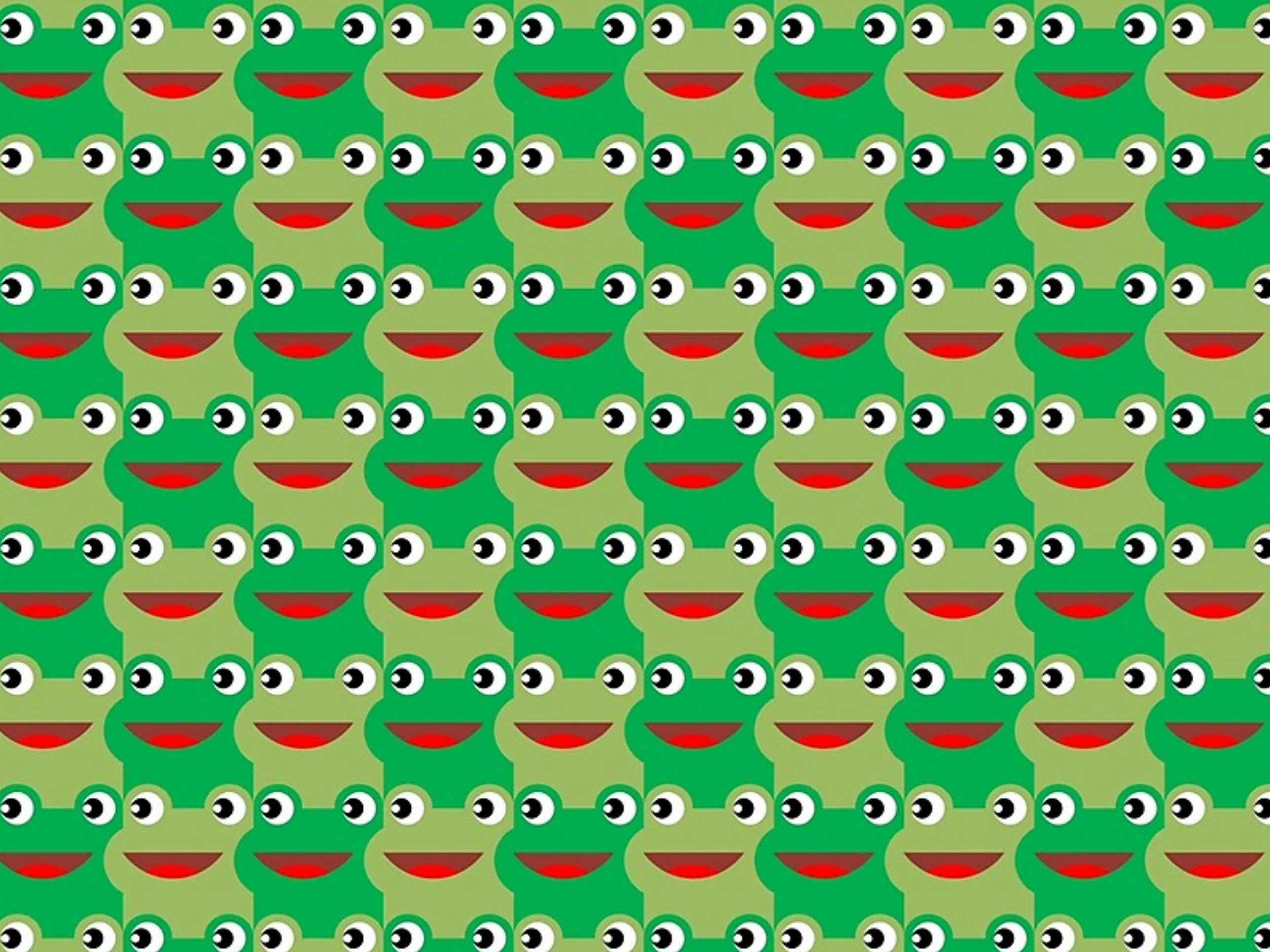
Book page 77

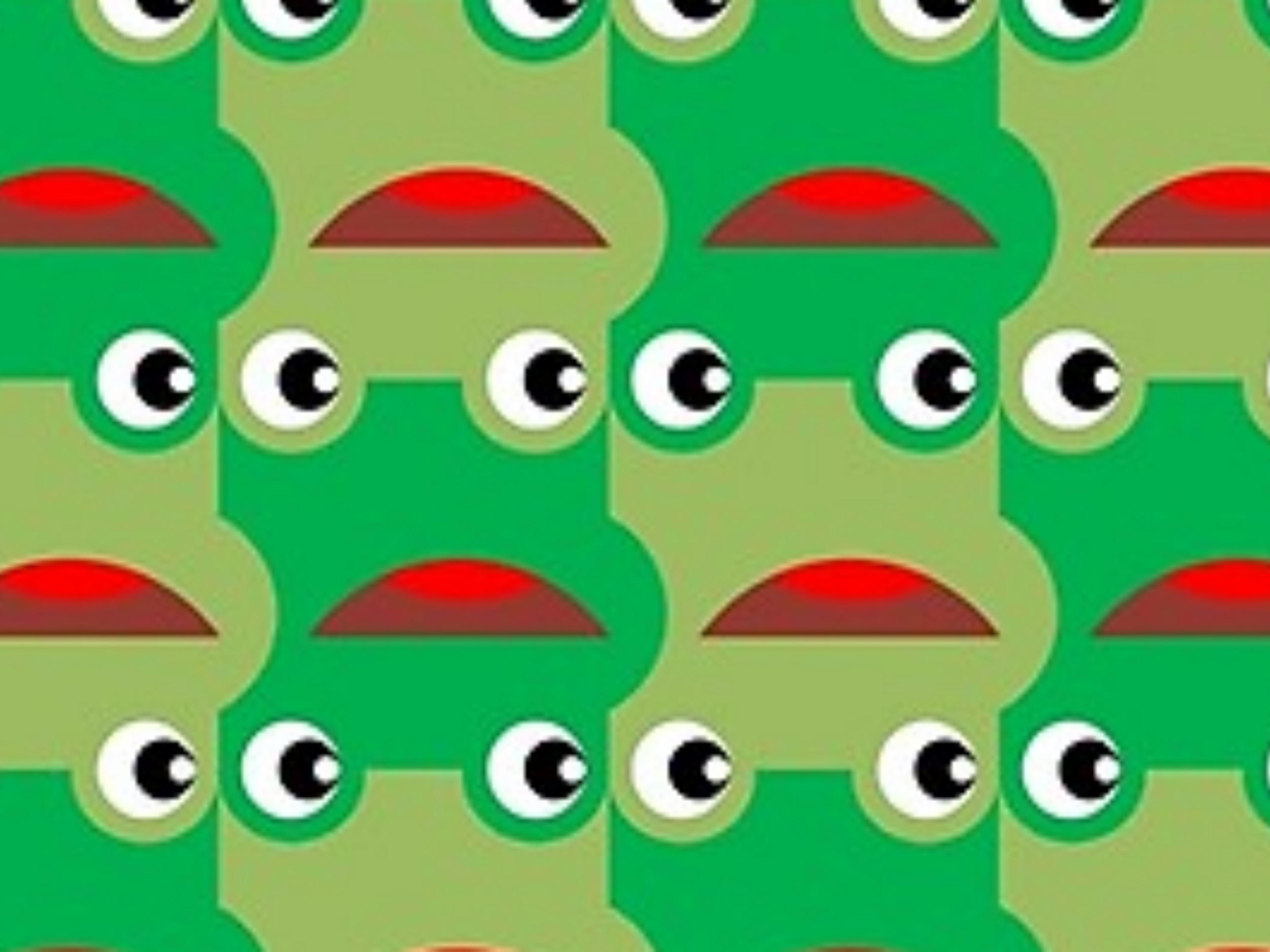
NOT - One's Complement Negation

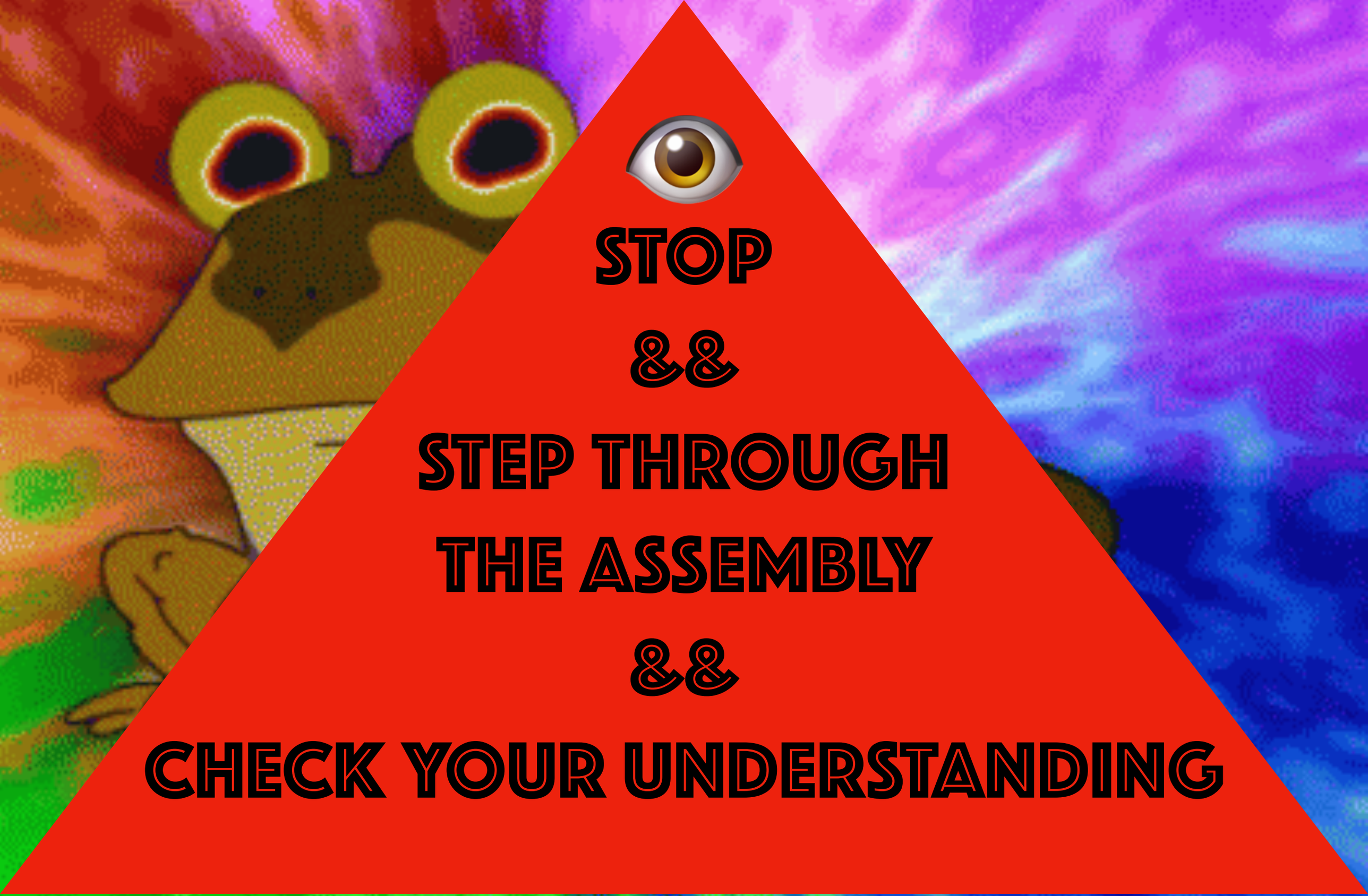
- C binary operator "~" (not !, that's *logical* NOT)
- Single source/destination operand can be r/mX

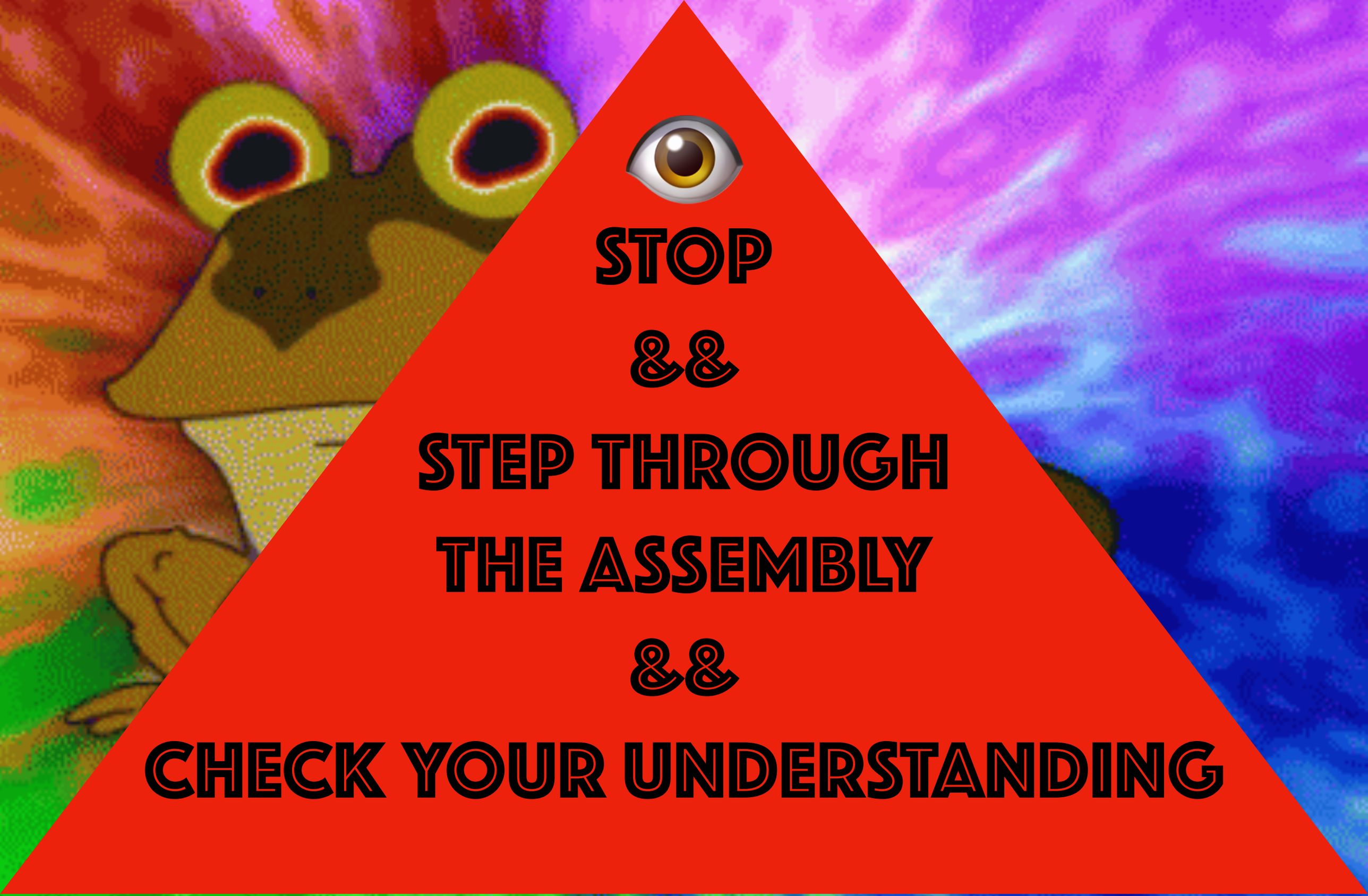
not rax

RAX	`1111100000011010101000011110111 0`01100000010010100011110101001010b
result	`0000011111100101010111100001000 1`01001111110110101110000101011010101b (RAX = 0x7E55E11A7ED70AD5!)






STOP
&&
STEP THROUGH
THE ASSEMBLY
&&
CHECK YOUR UNDERSTANDING


STOP
&&
STEP THROUGH
THE ASSEMBLY
&&
CHECK YOUR UNDERSTANDING

BooleanBecause.c Takeaways

- C binary operators correspond 1:1 to assembly instructions

```c
#include <stdio.h>
#define uint64 unsigned long long

unsigned long long main() {
    unsigned int i = 0x50da;
    unsigned int j = 0xc0ffee;
    uint64 k = 0x7ee707a11ed;
    k ^= ~(i & j) | 0x7ab00;
    return k;
}
```

```
main:
0000000140001000   sub   rsp,18h
0000000140001004   mov   dword ptr [rsp+4],50DAh
000000014000100C   mov   dword ptr [rsp],0C0FFEEh
0000000140001013   mov   rax,7EE707A11EDh
000000014000101D   mov   qword ptr [rsp+8],rax
0000000140001022   mov   eax,dword ptr [rsp]
0000000140001025   mov   ecx,dword ptr [rsp+4]
0000000140001029   and   ecx,eax
000000014000102B   mov   eax,ecx
000000014000102D   not   eax
000000014000102F   or    eax,7AB00h
0000000140001034   mov   eax,eax
0000000140001036   mov   rcx,qword ptr [rsp+8]
000000014000103B   xor   rcx,rax
000000014000103E   mov   rax,rcx
0000000140001041   mov   qword ptr [rsp+8],rax
0000000140001046   mov   rax,qword ptr [rsp+8]
000000014000104B   add   rsp,18h
000000014000104F   ret
```

Instructions we now know (19)

- NOP
- PUSH/POP
- CALL/RET
- MOV
- ADD/SUB
- IMUL
- MOVZX/MOVSX
- LEA
- JMP/Jcc (family)
- CMP
- **AND/OR/XOR/NOT**

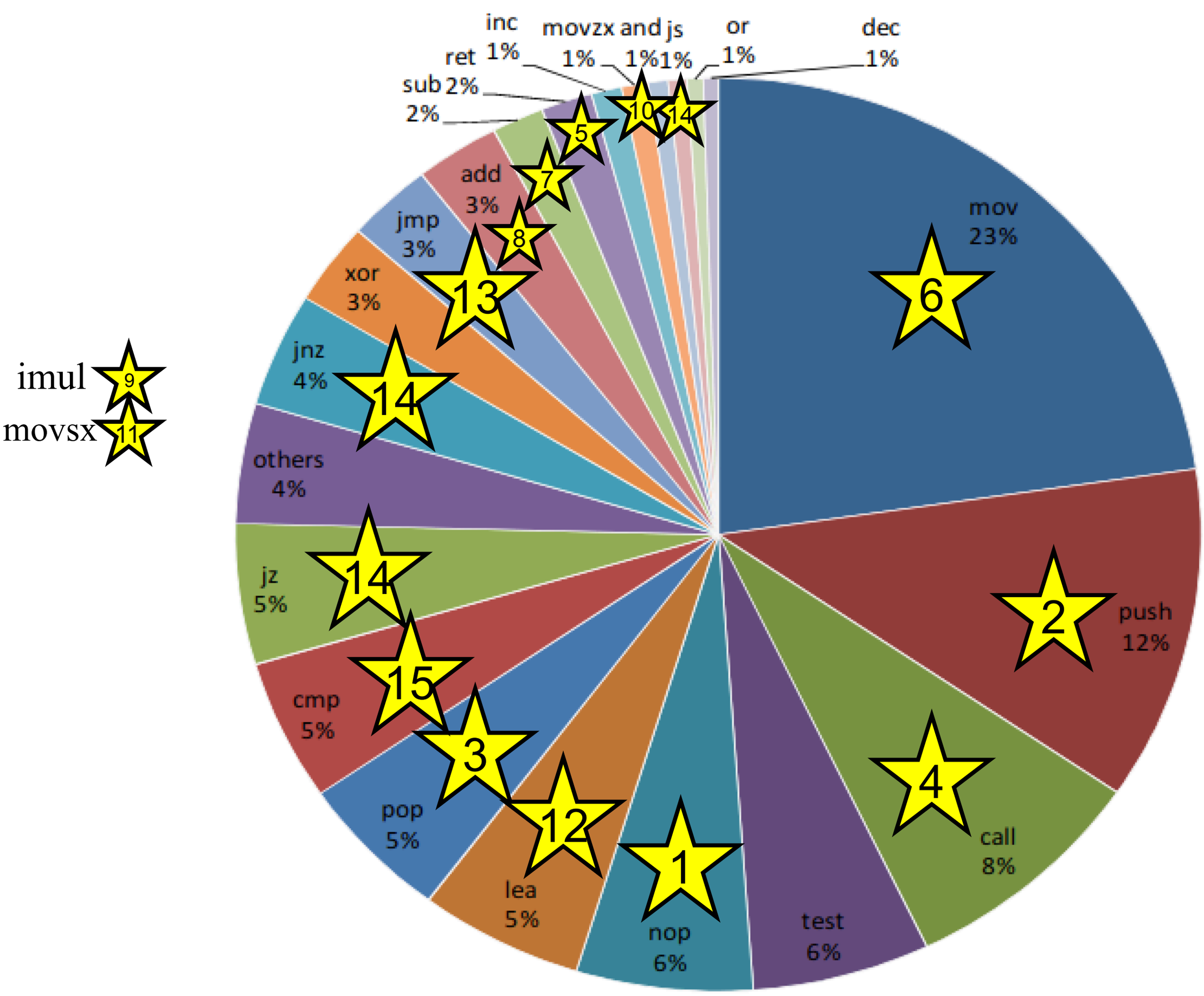


Fig. 2 Web browsers' instruction frequencies

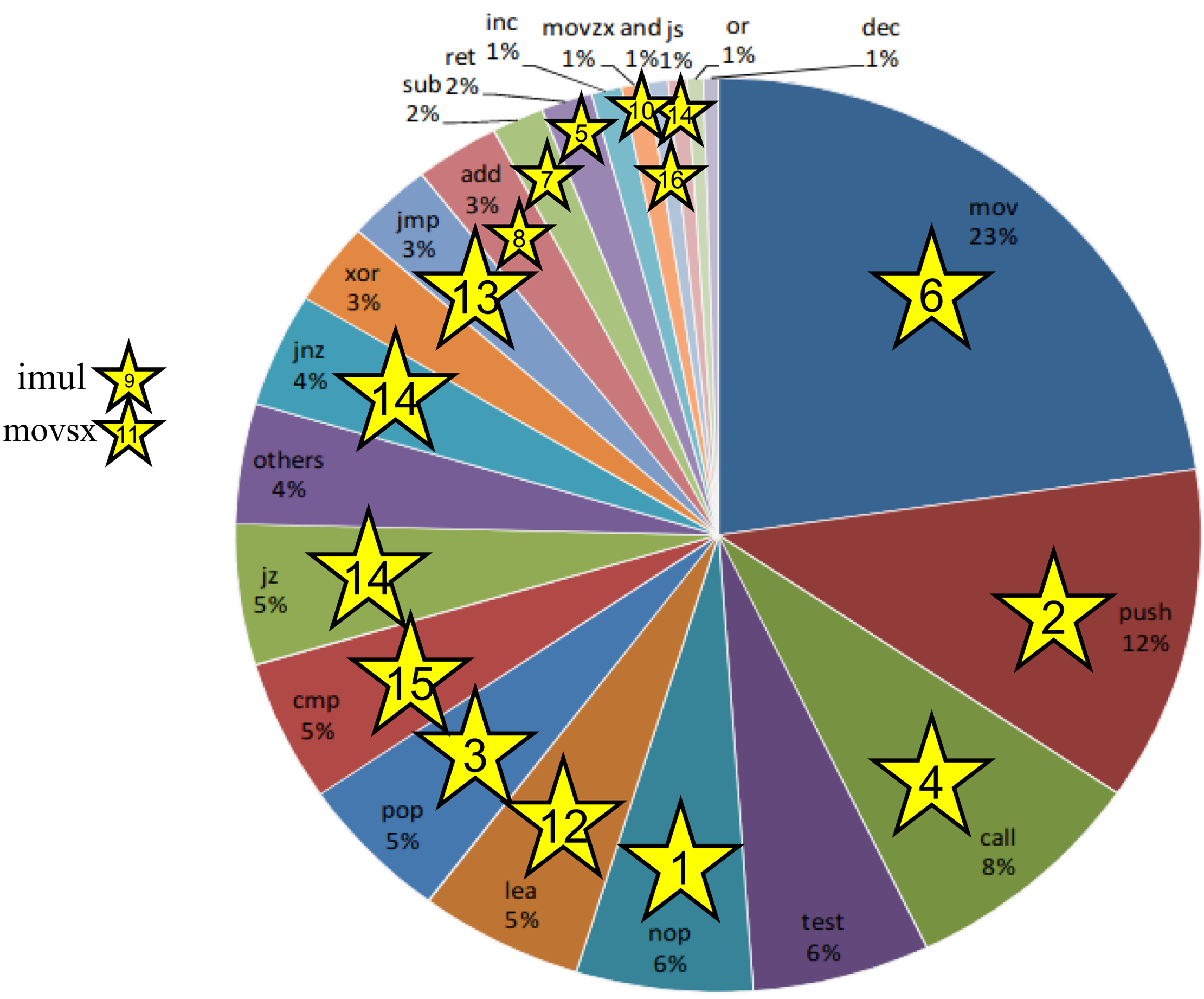


Fig. 2 Web browsers' instruction frequencies

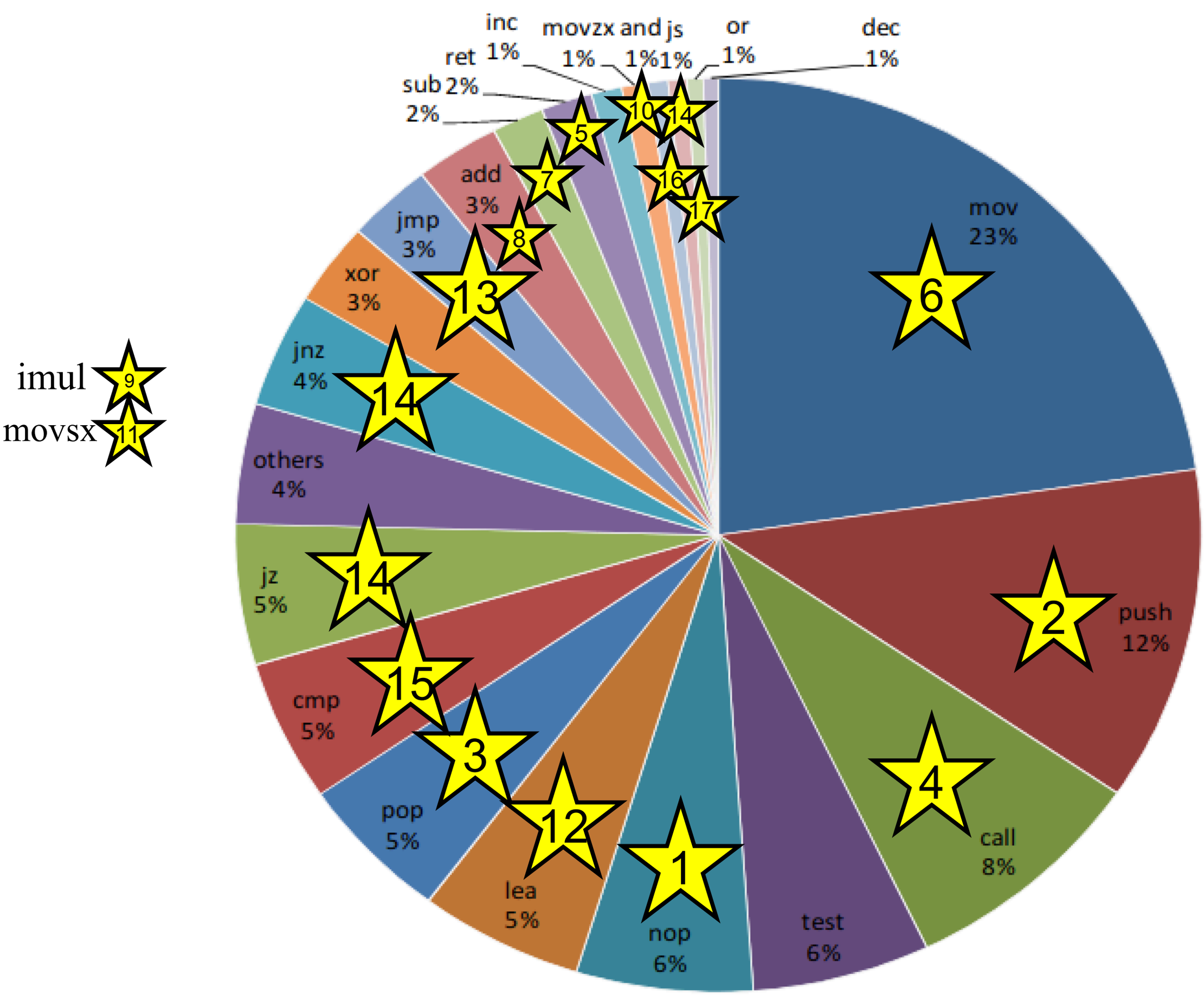


Fig. 2 Web browsers' instruction frequencies

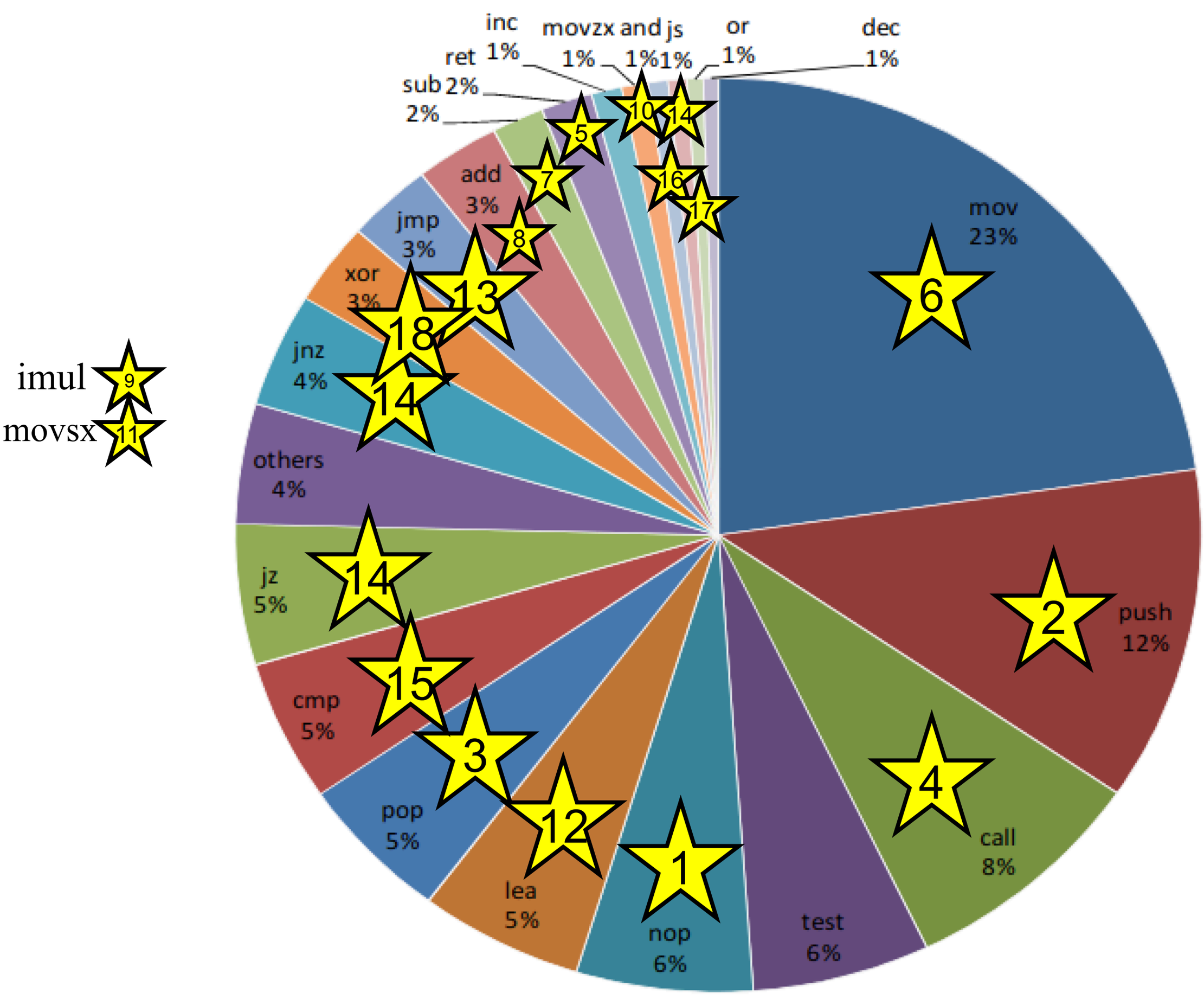


Fig. 2 Web browsers' instruction frequencies

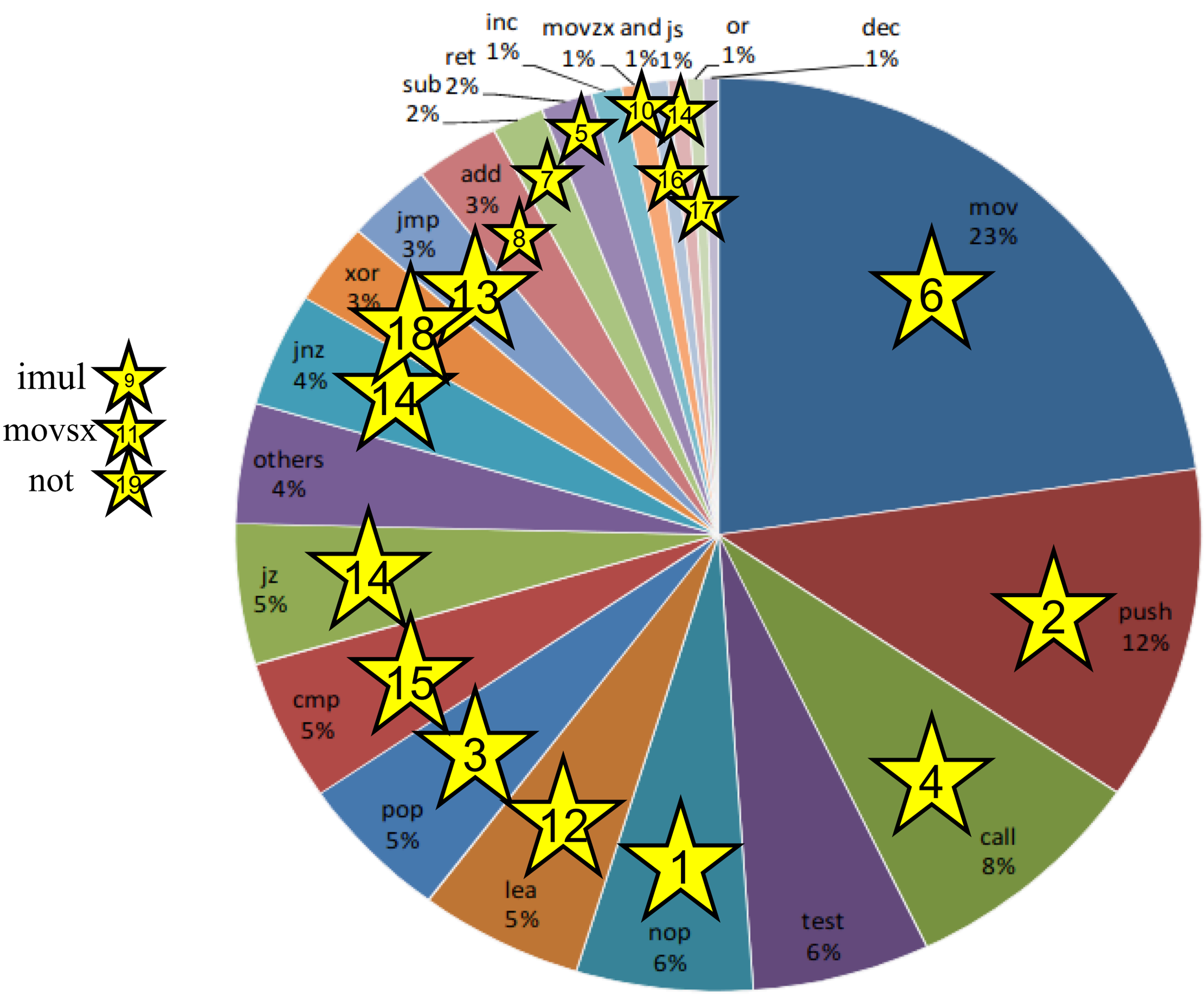


Fig. 2 Web browsers' instruction frequencies